Performance Testing

Date	30 June 2026
Team ID	SWTID-2026-4836
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Manual Browser Testing and Flask Development Server
Type of Testing	Load Testing and Functional Performance Testing
Target Module / API	Crop Prediction Module (/predict endpoint)
Test Environment	Local System (Windows 10, Flask Local Server)
Test Date	30 June 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user crop prediction request	1	30	Crop recommendation displayed successfully
2	Multiple prediction requests	10	60	System responds without errors
3	Continuous form submissions	20	120	Application remains stable
4	Invalid input testing	5	30	Input validation message is displayed

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.2 seconds	Pass	Fast prediction response
2	Response Time (Max)	< 5 seconds	2.8 seconds	Pass	Stable under multiple requests
3	Throughput (Req/sec)	> 5 Req/sec	8 Req/sec	Pass	Good request handling
4	Error Rate	< 1%	0%	Pass	No errors observed
5	CPU Utilization	< 80%	42%	Pass	Acceptable CPU usage
6	Memory Utilization	< 80%	48%	Pass	Memory usage within limits

Step 4: Observations & Analysis

Key Findings

The OptiCrop application successfully generated crop recommendations with low response time and stable performance. The system handled multiple prediction requests without failures.

Bottlenecks Identified

No major performance bottlenecks were identified during testing. Minor delays occurred when repeated requests were submitted continuously.

Optimization Steps Taken

- Optimized dataset loading and preprocessing.
- Loaded the trained model only once during application startup.
- Reduced unnecessary computations during prediction.
- Improved input validation and error handling.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

[Insert Screenshot 1 here] [Insert Screenshot 2 here]